

```
=== FILE: map_editor/ui/canvas.py ===
```

```
"""Отрисовка карты с текстурами (автоматический скейлинг + кэш)"""
```

```
import pygame
import os
from ..config import COLORS, SYMBOL_COLORS
from config.game_data import SYMBOLS_CONFIG, NPC_CONFIG
```

```
class Canvas:
```

```
    """Область отрисовки карты"""
```

```
    def __init__(self, rect):
```

```
        self.rect = rect
```

```
        self.grid = []
```

```
        self.width = 0
```

```
        self.height = 0
```

```
        self.cell_size = 20
```

```
        self.scroll_x = 0
```

```
        self.scroll_y = 0
```

```
        self.dragging = False
```

```
        self.drag_start_x = 0
```

```
        self.drag_start_y = 0
```

```
        self.scroll_start_x = 0
```

```
        self.scroll_start_y = 0
```

```
        self.hover_cell = None
```

```
        # Кэш текстур и шрифтов
```

```
        self.texture_cache = {}
```

```
        self.font_cache = {}
```

```
        self.ROOT_DIR = os.path.dirname(os.path.dirname(os.path.dirname(os.path.abspath(__file__))))
```

```
# =====
```

```
# ЗАГРУЗКА ТЕКСТУР (АВТОМАТИЧЕСКАЯ)
```

```
# =====
```

```
    def _get_texture(self, symbol):
```

```
        """Возвращает текстуру из кэша (для стен, предметов, NPC)"""
```

```
        if symbol in self.texture_cache:
```

```
            return self.texture_cache[symbol]
```

```
        texture_path = None
```

```
        # 1. Ищем в SYMBOLS_CONFIG
```

```
        config = SYMBOLS_CONFIG.get(symbol, {})
```

```
        texture_path = config.get('texture') or config.get('sprite')
```

```
        # 2. Если не нашли — ищем в NPC_CONFIG
```

```
        if not texture_path:
```

```
            npc_config = NPC_CONFIG.get(symbol, {})
```

```
            name = npc_config.get('name')
```

```
            if name:
```

```
                variants = [
```

```
                    f"{name}_move_front_1.png",
```

```

        f"{name}_idle_front.png",
        f"{name}_front.png",
    ]
    for variant in variants:
        test_path = os.path.join(self.ROOT_DIR, 'resources', 'npc', name, variant)
        if os.path.exists(test_path):
            texture_path = test_path
            break

    if not texture_path:
        folder_path = os.path.join(self.ROOT_DIR, 'resources', 'npc', name)
        if os.path.exists(folder_path):
            for file in os.listdir(folder_path):
                if file.endswith('.png'):
                    texture_path = os.path.join(folder_path, file)
                    break

# 3. Загружаем текстуры
if texture_path and os.path.exists(texture_path):
    try:
        tex = pygame.image.load(texture_path).convert_alpha()
        self.texture_cache[symbol] = tex
        return tex
    except Exception as e:
        print(f"[Canvas] Ошибка загрузки {texture_path}: {e}")

self.texture_cache[symbol] = None
return None

def _get_font(self, size):
    """Возвращает шрифт из кэша"""
    if size not in self.font_cache:
        self.font_cache[size] = pygame.font.Font(None, size)
    return self.font_cache[size]

# =====
# УПРАВЛЕНИЕ КАРТОЙ
# =====

def set_grid(self, grid):
    """Устанавливает карту для отрисовки"""
    self.grid = grid
    self.height = len(grid)
    self.width = len(grid[0]) if grid else 0
    self._center_view()

def _center_view(self):
    if self.width == 0 or self.height == 0:
        return

    max_cell_w = self.rect.width // self.width
    max_cell_h = self.rect.height // self.height
    self.cell_size = min(max_cell_w, max_cell_h, 60)
    self.cell_size = max(self.cell_size, 8)

    total_w = self.width * self.cell_size
    total_h = self.height * self.cell_size
    self.scroll_x = (self.rect.width - total_w) // 2
    self.scroll_y = (self.rect.height - total_h) // 2

```

```

def _clamp_scroll(self):
    if self.width == 0 or self.height == 0:
        return

    total_w = self.width * self.cell_size
    total_h = self.height * self.cell_size

    min_scroll_x = self.rect.width - total_w
    min_scroll_y = self.rect.height - total_h
    max_scroll_x = max(100, self.rect.width // 2)
    max_scroll_y = max(100, self.rect.height // 2)

    if total_w < self.rect.width:
        self.scroll_x = (self.rect.width - total_w) // 2
    else:
        self.scroll_x = max(min_scroll_x - 50, min(self.scroll_x, max_scroll_x))
        self.scroll_x = min(max_scroll_x, self.scroll_x)

    if total_h < self.rect.height:
        self.scroll_y = (self.rect.height - total_h) // 2
    else:
        self.scroll_y = max(min_scroll_y - 50, min(self.scroll_y, max_scroll_y))
        self.scroll_y = min(max_scroll_y, self.scroll_y)

# =====
# 3YM
# =====

def zoom_in(self):
    old_size = self.cell_size
    self.cell_size = min(self.cell_size + 2, 80)
    if self.cell_size != old_size:
        self._adjust_scroll_after_zoom(old_size)
        self._clamp_scroll()

def zoom_out(self):
    old_size = self.cell_size
    self.cell_size = max(self.cell_size - 2, 6)
    if self.cell_size != old_size:
        self._adjust_scroll_after_zoom(old_size)
        self._clamp_scroll()

def _adjust_scroll_after_zoom(self, old_size):
    ratio = self.cell_size / old_size
    self.scroll_x = self.scroll_x * ratio
    self.scroll_y = self.scroll_y * ratio

# =====
# РАБОТА С МЫШЬЮ
# =====

def get_cell_at(self, mouse_x, mouse_y):
    if not self.rect.collidepoint(mouse_x, mouse_y):
        return None

    rel_x = mouse_x - self.rect.x - self.scroll_x
    rel_y = mouse_y - self.rect.y - self.scroll_y

```

```

    if rel_x < 0 or rel_y < 0:
        return None

    grid_x = rel_x // self.cell_size
    grid_y = rel_y // self.cell_size

    if 0 <= grid_x < self.width and 0 <= grid_y < self.height:
        return (int(grid_x), int(grid_y))
    return None

def get_cell_rect(self, x, y):
    return pygame.Rect(
        self.rect.x + self.scroll_x + x * self.cell_size,
        self.rect.y + self.scroll_y + y * self.cell_size,
        self.cell_size,
        self.cell_size
    )

def start_drag(self, mouse_x, mouse_y):
    self.dragging = True
    self.drag_start_x = mouse_x
    self.drag_start_y = mouse_y
    self.scroll_start_x = self.scroll_x
    self.scroll_start_y = self.scroll_y

def update_drag(self, mouse_x, mouse_y):
    if self.dragging:
        dx = mouse_x - self.drag_start_x
        dy = mouse_y - self.drag_start_y
        self.scroll_x = self.scroll_start_x + dx
        self.scroll_y = self.scroll_start_y + dy
        self._clamp_scroll()

def end_drag(self):
    self.dragging = False

def update_hover(self, mouse_x, mouse_y):
    if not self.dragging:
        self.hover_cell = self.get_cell_at(mouse_x, mouse_y)

# =====
# ОТРИСОВКА
# =====

def draw(self, screen):
    """Отрисовывает карту с текстурами"""
    pygame.draw.rect(screen, COLORS['background'], self.rect)

    if not self.grid or self.width == 0:
        font = pygame.font.Font(None, 24)
        text = font.render("Загрузите уровень", True, COLORS['text_dim'])
        text_rect = text.get_rect(center=self.rect.center)
        screen.blit(text, text_rect)
        return

    # Видимая область
    start_x = max(0, int(-self.scroll_x // self.cell_size) - 1)
    start_y = max(0, int(-self.scroll_y // self.cell_size) - 1)
    end_x = min(self.width, int((self.rect.width - self.scroll_x) // self.cell_size) + 2)

```

```

end_y = min(self.height, int((self.rect.height - self.scroll_y) // self.cell_size) + 2)

font_size = max(8, int(self.cell_size * 0.4))

for y in range(start_y, end_y):
    for x in range(start_x, end_x):
        symbol = self.grid[y][x]
        rect = self.get_cell_rect(x, y)

        if rect.right < self.rect.x or rect.left > self.rect.right:
            continue
        if rect.bottom < self.rect.y or rect.top > self.rect.bottom:
            continue

        # 1. Текстура (автоматически для всего)
        texture = self._get_texture(symbol)

        if texture:
            scaled = pygame.transform.scale(texture, (rect.width, rect.height))
            screen.blit(scaled, rect)
        else:
            # 2. Цвет (fallback)
            color = SYMBOL_COLORS.get(symbol, (50, 50, 55))
            pygame.draw.rect(screen, color, rect)

            # 3. Текст (если есть что показать)
            if symbol != '_' and symbol != ' ':
                display_text = symbol[:7] + '...' if len(symbol) > 8 else symbol
                font = self._get_font(font_size)
                text = font.render(display_text, True, (255, 255, 255))
                text_rect = text.get_rect(center=rect.center)
                screen.blit(text, text_rect)

# Сетка
self._draw_grid(screen, start_x, start_y, end_x, end_y)

# Подсветка клетки под курсором
if self.hover_cell and not self.dragging:
    x, y = self.hover_cell
    if start_x <= x < end_x and start_y <= y < end_y:
        rect = self.get_cell_rect(x, y)
        pygame.draw.rect(screen, COLORS['selection'], rect, 2)

def _draw_grid(self, screen, start_x, start_y, end_x, end_y):
    for x in range(start_x, end_x + 1):
        pos = self.rect.x + self.scroll_x + x * self.cell_size
        if self.rect.x <= pos <= self.rect.right:
            pygame.draw.line(screen, COLORS['grid'], (pos, self.rect.y), (pos, self.rect.bottom), 1)

    for y in range(start_y, end_y + 1):
        pos = self.rect.y + self.scroll_y + y * self.cell_size
        if self.rect.y <= pos <= self.rect.bottom:
            pygame.draw.line(screen, COLORS['grid'], (self.rect.x, pos), (self.rect.right, pos), 1)

```